resource belongs to a customer the principal supports. These policies are straightforward and static, but they create dynamic behavior because the relationship graph is always changing.

This is what it means to treat policy as data: The policy logic remains constant, whereas access behavior adapts to the changing state of the graph. In PBAC, this kind of dynamic decision-making is made possible by treating relationships as structured input data. The logic is based on a few reusable rules; however, the access outcomes shift as relationships change. Anyone who can update the graph can affect authorization without needing to write code.

6.3.2 Policy as code

ABAC takes a different approach. In section 6.2, we examined how ACME Corp engineers controlled access to production tools using a set of policies that utilized contextual attributes. These policies were implemented as code, rules evaluated by a policy engine using current attributes as inputs. Unlike ReBAC, which grants access through relationship data, ABAC focuses on executable policies designed for precise evaluation.

This approach represents policies as code. Policies are created using languages like Cedar, Rego, or XACML and stored in one or more policy stores. They can be combined, layered, and managed using version control, just like any other codebase. Although more complex to create and maintain than relationship data, code-based policies provide unmatched accuracy, adaptability, and auditability.

Whether policies are implemented through relationship graphs or authored as executable rules, PBAC offers a shared architecture for authorization across systems. The next subsection explores how this hybrid approach is put into practice and how the models interact.

6.3.3 Beyond one-size-fits-all

Policy-based authorization provides a mental framework that helps identity teams unify the diverse approaches to access control across their systems. Instead of choosing between roles, attributes, or relationships, PBAC